

Parcial 1 — Liquidación de entregas en un centro de acopio de leche

Asignatura: Programación III — Universidad del Quindío

Docente: Julián E. Gutiérrez Posada

Lenguaje: Elixir (scripts `.exs`, sin `mix`)

Integrantes:

Integrante	Responsabilidad
Miguel Angel Ceballos Soler (Integrante 1)	Datos, validación y cálculos de la liquidación
Víctor Manuel Bolaños Guzmán (Integrante 2)	Reportes R1 a R8 y <code>ranking/2</code>
Juan José Ramírez Londoño (Integrante 3)	Interacción con el usuario, comprobante y orquestación (<code>main.exs</code>)

Ejecución: `elixir main.exs`

Parte A. Diseño

1. Colecciones utilizadas y por qué

Listas. Los datos llegan en listas (`Datos.productores/0`, `Datos.tanques/0`, `Datos.entregas/0`), tal como lo pide el enunciado. Las dejamos como listas porque casi todo el trabajo consiste en recorrerlas completas (filtrar, sumar, agrupar) con `Enum`, y para eso la lista es suficiente. Las entregas válidas y las rechazadas también se guardan en listas.

Mapas. Cada productor, tanque, entrega y liquidación es un mapa con claves atómicas (`entrega.litros`, `productor.codigo`). Se usan mapas porque permiten leer los campos por nombre, y no se pueden usar structs.

Además, en algunos puntos transformamos listas en mapas porque así el cálculo queda más claro:

Dónde	Transformación	Por qué
Datos.validas/0	lista de tuplas {"P01", "T1", 1, 250, 3.8} → lista de mapas con Enum.map	escribir 80 entregas como tuplas es más corto; el resto del programa trabaja con mapas
Reportes.agrupar_rechazadas/1 (R1)	Enum.group_by → %{motivo => [entregas]}	contar y listar los rechazos por motivo
Calculos.bonificacion_volumen_total/1	Enum.group_by(& &1.dia) → %{dia => [entregas]}	la bonificación depende del total por día
Reportes.litros_por_dia/1 (R3)	Enum.reduce → %{dia => litros}	es el mapa que después se combina con Map.merge/3
Reportes.ganador_semana/1 (R5)	Enum.frequencies_by → %{codigo => dias_ganados}	contar cuántos días ganó cada productor

Tuplas. Se usan para los resultados con estado ({:ok, entrega} / {:error, motivo}) y para devolver dos o tres valores juntos: clasificar_entregas/3 retorna {validas, rechazadas}, total_pagado/1 retorna {total, litros, costo_por_litro} y mayor_por_dia/1 retorna {dia, litros, ganadores}.

Keyword lists. Se usan en ranking/2 para pasar las opciones de ordenamiento (por: y orden:). Se explica en la Parte C.

2. Estructuras de control

Estructura	Dónde	Para qué
<code>with</code>	<code>Validacion.validar_entrega/3</code> , <code>Interaccion.parsear_entrega_adicional/1</code>	encadenar verificaciones en orden; la primera que falle devuelve su <code>{:error, motivo}</code>
Guardas en cláusulas (<code>when</code>)	<code>verificar_dia/1</code> , <code>verificar_litros/1</code> , <code>verificar_grasa/1</code> , <code>Calculos.factor_grasa/1</code> , <code>Calculos.bonificacion_dia/1</code>	elegir la regla según el valor (rangos de grasa, límites de litros)
<code>case</code>	<code>main.exs</code> (entrega adicional), <code>Reportes.r6/1</code> , <code>r5/1</code> , <code>r8/3</code> , <code>parse_integer/1</code>	decidir según el resultado (<code>{:ok, _}</code> , <code>{:error, _}</code> , <code>:omitir</code> , lista vacía)
<code>if</code>	<code>verificar_productor/2</code> , <code>verificar_tanque/2</code> , <code>descuento_transporte_total/2</code>	condiciones simples de verdadero/falso
<code>Enum</code>	todo el programa	recorrer colecciones: <code>map</code> , <code>filter</code> , <code>reduce</code> , <code>group_by</code> , <code>sum</code> , <code>any?</code> , <code>all?</code> , <code>each</code>
Comprehensions <code>for</code>	<code>Reportes.mayor_por_dia/1</code> , <code>Reportes.ganador_semana/1</code>	quedarse con todos los códigos empatados en el máximo

No usamos recursividad, `defstruct` , `File` , procesos ni `try/rescue` . Los errores se manejan solo con tuplas.

Nota: `Util2.ex` es el módulo de utilidades del curso (autor: el docente). De él solo usamos `mostrar/2` , `ordenar/3` y `convertir_coleccion_mensaje/2` . Las funciones de `Util2` que leen colecciones desde el teclado (que internamente usan recursión) no se llaman en el proyecto.

3. Funciones puras e impuras

Una función **pura** siempre devuelve lo mismo para la misma entrada y no imprime ni lee nada. Una función **impura** hace entrada/salida (`IO.puts` , `IO.gets`) o mide el tiempo.

Módulo	Puras	Impuras
Datos	todas	—
Validacion	todas (validar_entrega/3 , clasificar_entregas/3 , verificaciones)	—
Calculos	todas (valor_entrega/1 , factor_grasa/1 , bonificacion_dia/1 , liquidacion_general/2 , ...)	—
Investigacion	combinar_centros/2 , combinar_sin_sumar/2	—
Reportes	las que calculan : agrupar_rechazadas/1 , ocupacion_tanques/2 , litros_por_dia/1 , mayor_por_dia/1 , ganador_semana/1 , grasa_ponderada/1 , calidad_productores/1 , total_pagado/1 , en_todos_los_tanques/3 , ranking/2	las que muestran : r1/1 ... r8/3 y generar/5
Interaccion	parsear_entrega_adicional/1	solicitar_entrega_adicional/0 , mostrar_comprobante/3
Main	—	ejecutar/0 , agregar_entrega_adicional/4 , medir_promedio/1

En `Reportes` cada reporte está partido en dos: una función pura que calcula los datos y una función `rx` que solo los imprime. Así los cálculos se pueden probar sin mirar la consola. Por ejemplo, `main.exs` reutiliza `Reportes.litros_por_dia/1` para la investigación sin volver a imprimir R3.

4. Parámetros como atributos de módulo

Parámetro	Atributo
Tarifa base \$1.800	Calculos @tarifa_base
Litros para bonificación 450	Calculos @litros_min_volumen
Bonificación \$25.000	Calculos @bonificacion_volumen
Transporte \$18.000	Calculos @costo_transporte
Días 1 a 6	Validacion @dias , Reportes @dias
Máximo 800 L	Validacion @max_litros
Grasa entre 0 y 15	Validacion @grasa_minima , @grasa_maxima
Meta diaria 2.000 L	Reportes @meta_diaria

Parte B. Especificación de los módulos

datos.exs — **Datos** . 10 productores (4 con transporte), 4 tanques y 90 entregas: 80 válidas en los 6 días y 10 inválidas (2 por cada motivo de rechazo).

validacion.exs — **Validacion** .

- `validar_entrega(entrega, productores, tanques)` → `{:ok, entrega}` o `{:error, motivo}` . Verifica con `with` , en este orden: productor, tanque, día, litros y grasa. Solo se informa el primer motivo.
- `clasificar_entregas(entregas, productores, tanques)` → `{validas, rechazadas}` , donde cada rechazada es `%{entrega: entrega, motivo: motivo}` .

calculos.exs — **Calculos** .

- `valor_entrega(entrega)` = `litros × 1800 × (1 + factor_grasa(grasa))` .
- `factor_grasa(grasa)` : 0.06 si ≥ 3.5 ; 0.0 si ≥ 3.0 ; -0.08 si ≥ 2.5 ; -0.20 si es menor.
- `bonificacion_dia(litros_dia)` : \$25.000 si el día suma ≥ 450 L, si no \$0. La usan la liquidación y el comprobante.
- `liquidacion_general(productores, validas)` → una liquidación por productor, con las claves `:codigo`, `:nombre`, `:transporte`, `:entregas_count`, `:litros_totales`, `:pago_bruto`, `:bonificacion_volumen`, `:descuento_transporte`, `:pago_netto` . Un productor sin entregas válidas aparece con todo en cero.

reportes.exs — **Reportes** (Integrante 2).

Reporte	Función que calcula (pura)	Qué retorna
R1	<code>agrupar_rechazadas(rechazadas)</code>	<code>%{motivo => [entregas]}</code> ; <code>r1</code> recorre los 5 motivos, incluidos los que tienen 0 rechazos
R2	<code>ocupacion_tanques(validas, tanques)</code>	cada tanque con <code>:litros</code> y <code>:ocupacion (%)</code> ; un tanque sin entregas queda en 0. Se ordena con <code>ranking(por: :ocupacion, orden: :desc)</code>
R3	<code>litros_por_dia(validas)</code> y <code>cumple_meta?(litros)</code>	<code>%{dia => litros}</code> con los 6 días; al final usa <code>Enum.all?</code> y <code>Enum.any?</code>
R4	<code>ranking(liquidacion, por: :pago_netto, orden: :desc)</code>	liquidación ordenada; <code>Enum.with_index(1)</code> la numera
R5	<code>mayor_por_dia(validas)</code> y <code>ganador_semana(mayores)</code>	<code>{dia, litros_max, [ganadores]}</code> (con empates) y <code>{[codigos], dias}</code>
R6	<code>calidad_productores(validas)</code> , <code>grasa_ponderada/1</code> , <code>grasa_simple/1</code>	productores con ≥ 3 entregas, con promedio ponderado y simple
R7	<code>total_pagado(liquidacion)</code>	<code>{total, litros, costo_por_litro}</code> ; si no hay litros, el costo es 0
R8	<code>en_todos_los_tanques(validas, productores, tanques)</code>	productores que, para todos los tanques (<code>Enum.all?</code>), tienen alguna entrega (<code>Enum.any?</code>)

`interaccion.exs` — `Interaccion`.

- `solicitar_entrega_adicional()` lee `productor;tanque;dia;litros;grasa`. Con Enter vacío retorna `:omitir`.
- `parsear_entrega_adicional(cadena)` → `{:ok, entrega}` o `{:error, :formato_invalido}` si no hay 5 campos o si el día, los litros o la grasa no son números. Usa `Integer.parse/1` y `Float.parse/1`, sin `try/rescue`.
- `mostrar_comprobante(productores, validas, liquidaciones)` imprime el comprobante; si el código no existe, lo informa sin que el programa falle.

`investigacion.exs` — `Investigacion`. `combinar_centros/2` (con `Map.merge/3`) y `combinar_sin_sumar/2` (con `Map.merge/2`, solo para comparar).

`main.exs` — `Main`. Orden de ejecución: cargar los datos → validarlos → pedir la entrega adicional y validarla con las mismas reglas → liquidar → medir los tiempos → R1 a R8 → comprobante → investigación.

Explicación de R6: promedio ponderado frente a promedio simple

El **promedio simple** suma los porcentajes y divide por el número de entregas, así que **cada entrega pesa lo mismo** sin importar cuántos litros tenga:

```
simple = suma(grasa) / número de entregas
```

El **promedio ponderado** multiplica cada porcentaje por sus litros. Una entrega de 700 L pesa 700 veces más que una de 1 L:

```
ponderado = suma(grasa × litros) / suma(litros)
```

El ponderado representa la grasa real de **toda la leche** que entregó el productor, porque toda esa leche termina mezclada en los tanques. Los dos promedios solo coinciden cuando las entregas tienen litros parecidos. Si una entrega es mucho más grande que las demás, el ponderado se acerca a la grasa de esa entrega.

Caso de nuestros datos: P09 (Diana Beltrán).

Día	Tanque	Litros	Grasa
1	T3	700	2.6
2	T1	60	4.2
3	T3	60	4.2
4	T1	60	4.2
5	T1	60	4.2
6	T3	60	4.2

- Simple: $(2.6 + 4.2 \times 5) / 6 = 23.6 / 6 = \mathbf{3.933 \%}$
- Ponderado: $(700 \times 2.6 + 300 \times 4.2) / 1000 = (1820 + 1260) / 1000 = \mathbf{3.080 \%}$

Con el promedio simple, P09 sería **el mejor productor** de la semana (3.933 % es más que el 3.79 % de P06). Pero el 70 % de su leche (700 de 1000 L) tiene 2.6 % de grasa. Con el ponderado, P09 queda en el **puesto 7 de 10**. El ponderado no deja que cinco entregas pequeñas escondan una entrega grande de baja calidad. Por eso R6 usa el ponderado y el ganador es **P06 con 3.787 %**.

En el resto de productores la diferencia es pequeña (por ejemplo, P04: 3.434 % frente a 3.45 %) porque sus entregas tienen volúmenes parecidos.

Parte C. Investigación y mediciones

1. Keyword lists en `ranking/2`

Una keyword list es una lista de tuplas `{átomo, valor}`. Elixir permite escribirla de forma corta cuando es el último argumento de una función:

```
ranking(liquidacion, por: :pago_netto, orden: :desc)
# es lo mismo que
ranking(liquidacion, [{:por, :pago_netto}, {:orden, :desc}])
```

Nuestra implementación (`Reportes.ranking/2`):

```
def ranking(lista, opciones) do
  campo = Keyword.get(opciones, :por)
  orden = Keyword.get(opciones, :orden, :desc)

  Util2.ordenar(lista, orden, &Map.get(&1, campo))
end
```

- `Keyword.get(opciones, :por)` lee el campo por el que se ordena.
- `Keyword.get(opciones, :orden, :desc)` lee el sentido. Si no se envía, toma `:desc` **por defecto**.
- `&Map.get(&1, campo)` saca ese campo de cada mapa y `Util2.ordenar/3` (que usa `Enum.sort_by`) ordena.

Usos en el proyecto:

```
ranking(tanques, por: :ocupacion, orden: :desc) # R2
ranking(liquidacion, por: :pago_netto, orden: :desc) # R4
ranking(calidad, por: :ponderada, orden: :desc) # R6
```

¿Por qué keyword list y no un mapa o varios argumentos?

- Es la forma estándar en Elixir para pasar **opciones**, como hacen `Enum.sort_by/3` y `String.split/3`.

- La llamada se lee sola: `por: :pago_netto, orden: :desc` dice qué hace sin tener que recordar el orden de los argumentos.
- Permite **valores por defecto** con `Keyword.get/3: ranking(lista, por: :litros)` ordena de mayor a menor sin escribir `orden: .`
- Se pueden agregar opciones nuevas (por ejemplo `limite: .`) sin cambiar la firma de la función ni las llamadas existentes.

Un mapa también serviría, pero obligaría a escribir `%{por: ..., orden: ...}` en cada llamada. Las keyword lists además conservan el orden y permiten claves repetidas, lo que tiene sentido para opciones y no para datos.

2. `Map.merge/2` frente a `Map.merge/3`

Mapa de R3 de nuestra ejecución (con la entrega adicional `P03;T2;4;320.5;3.6`) y mapa del centro vecino:

```
mapa_r3 = %{1 => 3110, 2 => 2370, 3 => 2430, 4 => 2830.5, 5 => 1990, 6 => 2660}
centro_vecino = %{1 => 1850.5, 2 => 2100, 3 => 1640, 5 => 2350, 7 => 800}
```

Nuestra función:

```
def combinar_centros(mapa_r3, centro_vecino) do
  Map.merge(mapa_r3, centro_vecino, fn _dia, litros_r3, litros_vecino ->
    litros_r3 + litros_vecino
  end)
end
```

Resultados reales que imprime el programa:

```
Con Map.merge/2: %{1 => 1850.5, 2 => 2100, 3 => 1640, 4 => 2830.5, 5 => 2350, 6 => 2660, 7 => 800}
Con Map.merge/3: %{1 => 4960.5, 2 => 4470, 3 => 4070, 4 => 2830.5, 5 => 4340, 6 => 2660, 7 => 800}
```

1. ¿Qué habría pasado con `Map.merge/2`? Cuando una clave está en los dos mapas, `Map.merge/2` se queda con el valor del **segundo** mapa. En los días 1, 2, 3 y 5 el valor del centro vecino reemplaza al de R3: el día 1 queda con 1850.5 L y se pierden los 3110 L de nuestro centro.

2. ¿Por qué no es adecuado? Lo que se busca es el **total** recibido por los dos centros. Con `Map.merge/2` se pierden $3110 + 2370 + 2430 + 1990 = 9900$ L de nuestro centro. Además, el resultado sale como si fuera correcto (no hay ningún error), así que el problema no se nota. `Map.merge/3` llama a la función anónima en cada clave repetida y suma los dos valores.

3. ¿Qué pasa con el día 7? Solo está en el mapa del centro vecino. Como no hay choque de claves, `Map.merge/3` **no llama** a la función y copia `7 => 800` tal cual. Lo mismo pasa con los días 4 y 6, que solo están en R3 y se conservan con su valor. El resultado tiene la unión de todos los días.

3. Mediciones con `:timer.tc/1`

Qué se midió. Por separado:

- la validación de las 90 entregas iniciales (`Validacion.clasificar_entregas/3`);
- la liquidación de los 10 productores (`Calculos.liquidacion_general/2`).

Problema encontrado. Al medir una sola ejecución, `:timer.tc/1` devolvía **0 µs o 102 µs** en el equipo de pruebas (Windows 11). La operación tarda menos que la resolución del reloj del sistema, así que el número no significaba nada. La solución fue repetir cada operación **1000 veces** dentro de una sola medición y dividir el total por 1000:

```
defp medir_promedio(funcion) do
  {tiempo_total, :ok} = :timer.tc(fn -> Enum.each(1..@repeticiones, fn _ -> funcion.() end) end)
  tiempo_total / @repeticiones
end
```

Resultados (5 ejecuciones de `elixir main.exs`, Elixir 1.20.3 / OTP 29, Windows 11):

Ejecución	Validación (90 entregas)	Liquidación (10 productores)
1	21.30 µs	33.89 µs
2	22.43 µs	33.69 µs
3	22.32 µs	33.08 µs
4	23.04 µs	37.48 µs
5	21.71 µs	33.89 µs
Promedio	≈ 22.2 µs	≈ 34.4 µs

Explicación.

- **Validación:** recorre las $N = 90$ entregas una vez. Para cada una, `verificar_productor` y `verificar_tanque` usan `Enum.any?` sobre los $P = 10$ productores y los $T = 4$ tanques, así que en el peor caso hace $N \times (P + T)$ comparaciones: **$O(N \cdot (P + T))$** .
- **Liquidación:** por cada uno de los P productores, `Enum.filter` recorre **todas** las entregas válidas (≈ 81), así que el costo es **$O(P \cdot N)$** . Además, por cada entrega hace multiplicaciones con decimales, `group_by` por día y `uniq` de días.
- La liquidación tarda más aunque procesa "solo 10 productores", porque en realidad recorre $10 \times 81 \approx 810$ entregas y hace más trabajo con cada una.
- Con datos de este tamaño los dos tiempos son de microsegundos y no afectan al usuario. Si el centro tuviera miles de productores, la liquidación crecería más rápido. Se podría mejorar agrupando las entregas por productor una sola vez con `Enum.group_by` antes de liquidar.
- La ejecución 4 salió más lenta en la liquidación. Eso muestra que hay variación entre corridas (otros procesos del sistema operativo), y por eso conviene repetir y promediar.

Salida completa del programa

Entrada usada: la entrega adicional del enunciado `P03;T2;4;320.5;3.6` y el comprobante del productor `P09`.

```
=====
SISTEMA DE ACOPIO DE LECHE - PROGRAMACIÓN III
=====
```

```
=====
REGISTRO DE ENTREGA ADICIONAL
=====
```

Ingrese una entrega adicional (productor;tanque;dia;litros;grasa)
o Enter para omitir: Entrega adicional válida. Se incorpora a todos los reportes.

```
-----
Promedio de 1000 repeticiones medidas con :timer.tc/1
Validación (90 entregas): 22.84 µs
Liquidación (10 productores): 36.04 µs
-----
```

===== R1. Entregas rechazadas =====

productor_desconocido: 2 rechazo(s)
- P99, T1, día 1, 200 L, grasa 3.6
- PX0, T2, día 2, 150 L, grasa 3.2

tanque_desconocido: 2 rechazo(s)
- P01, T99, día 1, 180 L, grasa 3.4
- P02, TX4, día 3, 210 L, grasa 3.1

dia_invalido: 2 rechazo(s)
- P01, T1, día 0, 200 L, grasa 3.8
- P03, T2, día 7, 250 L, grasa 3.5

litros_fuera_de_rango: 2 rechazo(s)
- P02, T1, día 2, 0 L, grasa 3.2
- P04, T3, día 4, 850 L, grasa 3.6

porcentaje_invalido: 2 rechazo(s)
- P05, T2, día 3, 190 L, grasa -0.5
- P06, T4, día 5, 220 L, grasa 15.5

Total rechazadas: 10

===== R2. Litros por tanque =====

- T2 Tanque Central: 3890.5 L de 4000 L (97.3 %)
- T1 Tanque Norte: 4760 L de 5000 L (95.2 %)
- T4 Tanque Reserva: 2830 L de 3000 L (94.3 %)
- T3 Tanque Sur: 3910 L de 4500 L (86.9 %)

===== R3. Litros por día (meta 2000 L) =====

- Día 1: 3110 L -> cumple
- Día 2: 2370 L -> cumple
- Día 3: 2430 L -> cumple
- Día 4: 2830.5 L -> cumple
- Día 5: 1990 L -> no cumple
- Día 6: 2660 L -> cumple

¿Cumplió todos los días? No
¿Cumplió al menos un día? Sí

===== R4. Liquidación (por neto, mayor a menor) =====

1. P03 Ana Rodríguez | litros: 3080.5 | entregas: \$5877594 | bonif: \$125000 | transporte: \$108000 | neto: \$5894594
2. P01 Marta Gómez | litros: 2840 | entregas: \$5418720 | bonif: \$125000 | transporte: \$108000 | neto: \$5435720
3. P06 Carlos Ospina | litros: 1800 | entregas: \$3434400 | bonif: \$0 | transporte: \$108000 | neto: \$3326400
4. P04 Jorge Valencia | litros: 1700 | entregas: \$3142080 | bonif: \$0 | transporte: \$108000 | neto: \$3034080
5. P02 Luis Cardona | litros: 1170 | entregas: \$2106000 | bonif: \$0 | transporte: \$0 | neto: \$2106000
6. P08 Mario Ríos | litros: 1070 | entregas: \$1926000 | bonif: \$0 | transporte: \$0 | neto: \$1926000
7. P09 Diana Beltrán | litros: 1000 | entregas: \$1731600 | bonif: \$25000 | transporte: \$0 | neto: \$1756600
8. P05 Sofía Jaramillo | litros: 990 | entregas: \$1639440 | bonif: \$0 | transporte: \$0 | neto: \$1639440
9. P10 Gonzalo Patiño | litros: 930 | entregas: \$1563120 | bonif: \$0 | transporte: \$0 | neto: \$1563120
10. P07 Elena Morales | litros: 810 | entregas: \$1224720 | bonif: \$0 | transporte: \$0 | neto: \$1224720

===== R5. Mayor productor por día =====

- Día 1: P09 con 700 L
- Día 2: P01, P03 con 460 L
- Día 3: P03 con 460 L
- Día 4: P03 con 790.5 L
- Día 5: P01 con 440 L
- Día 6: P03 con 500 L

Ganador de la semana: P03 (4 días en primer lugar)

===== R6. Calidad (grasa ponderada, mínimo 3 entregas) =====

- P06: 10 entregas | ponderada 3.787 % | simple 3.79 %
- P01: 13 entregas | ponderada 3.725 % | simple 3.715 %
- P03: 14 entregas | ponderada 3.718 % | simple 3.714 %
- P04: 8 entregas | ponderada 3.434 % | simple 3.45 %
- P08: 6 entregas | ponderada 3.166 % | simple 3.167 %
- P02: 6 entregas | ponderada 3.099 % | simple 3.1 %
- P09: 6 entregas | ponderada 3.08 % | simple 3.933 %
- P10: 6 entregas | ponderada 2.885 % | simple 2.883 %
- P05: 6 entregas | ponderada 2.752 % | simple 2.75 %
- P07: 6 entregas | ponderada 2.417 % | simple 2.417 %

Mejor calidad: P06 con 3.787 %

===== R7. Total pagado =====

Total pagado en la semana: \$27906674

Litros pagados: 15390.5 L

Costo promedio por litro: \$1813.24

===== R8. Productores que entregaron en los 4 tanques =====

- P01 Marta Gómez
- P02 Luis Cardona
- P03 Ana Rodríguez
- P04 Jorge Valencia
- P05 Sofía Jaramillo
- P06 Carlos Ospina
- P07 Elena Morales
- P08 Mario Ríos

=====

COMPROBANTE DE PRODUCTOR

=====

Ingrese el código del productor (ej. P01):

COMPROBANTE FINANCIERO - CENTRO DE ACOPIO DE LECHE

```
-----
Productor: Diana Beltrán (P09)
Usa Servicio de Transporte: NO
-----
DETALLE POR DÍA CON ENTREGAS VÁLIDAS:
  • Día 1: 700 L | Valor Entregas: $1159200 | Bonificación Día: $25000
  • Día 2: 60 L | Valor Entregas: $114480 | Bonificación Día: $0
  • Día 3: 60 L | Valor Entregas: $114480 | Bonificación Día: $0
  • Día 4: 60 L | Valor Entregas: $114480 | Bonificación Día: $0
  • Día 5: 60 L | Valor Entregas: $114480 | Bonificación Día: $0
  • Día 6: 60 L | Valor Entregas: $114480 | Bonificación Día: $0
-----
RESUMEN GENERAL DE LA SEMANA:
  Total Entregas Válidas: 6
  Total Litros Entregados: 1000 L
  Valor Bruto Entregas: $1731600
  (+) Bonif. Volumen: $25000
  (-) Descuento Transp: $0
-----
  NETO A PAGAR: $1756600
-----

=====
      INVESTIGACIÓN: COMBINACIÓN DE CENTROS
=====
Mapa R3 (Centro Principal): %{1 => 3110, 2 => 2370, 3 => 2430, 4 => 2830.5, 5 => 1990, 6 => 2660}
Mapa Centro Vecino:      %{1 => 1850.5, 2 => 2100, 3 => 1640, 5 => 2350, 7 => 800}
Con Map.merge/2:         %{1 => 1850.5, 2 => 2100, 3 => 1640, 4 => 2830.5, 5 => 2350, 6 => 2660, 7 =>
800}
Con Map.merge/3:         %{1 => 4960.5, 2 => 4470, 3 => 4070, 4 => 2830.5, 5 => 4340, 6 => 2660, 7 =>
800}
=====
```

Entradas inválidas probadas (el programa no falla en ningún caso):

Entrada	Resultado
Enter vacío	Entrega adicional omitida.
P03;T2;4 (3 campos)	[ERROR] Formato de entrega inválido.
P03;T2;cuatro;300;3.5 (día no entero)	[ERROR] Formato de entrega inválido.
P03;T2;4;abc;3.5 (litros no numéricos)	[ERROR] Formato de entrega inválido.
P03;T2;4;300;alta (grasa no numérica)	[ERROR] Formato de entrega inválido.
P99;T2;4;300;3.5 (formato bien, productor no existe)	rechazada por productor_desconocido ; R1 muestra 11 rechazadas
P03;T2;4;900;3.5 (más de 800 L)	rechazada por litros_fuera_de_rango ; R1 muestra 11 rechazadas
Comprobante con código X	[ERROR] El código de productor 'X' no existe en el sistema.

Parte D. Bitácora de IA, caso de error y reflexión

1. Bitácora de uso de IA

Fecha	Integrante	Herramienta	Para qué se usó	Qué se hizo con la respuesta
2026-10-01	Victor Bolaños	Claude Code	Preguntar cuáles reportes eran los más complejos (R5, R6, R8) para preparar la sustentación	Se revisó contra el código de reportes.exs ; se usó como guía de estudio
2026-10-01	Victor Bolaños	Claude Code	Revisar este documento contra el código y contra el enunciado	Se encontraron datos inventados (R3, tiempos), la interfaz vieja de ranking/2 y partes faltantes (R6, keyword lists, salida, Parte D)
2026-10-01	Victor Bolaños	Claude Code	Corregir datos y código: atributos de módulo, encabezados con integrantes, orden de la entrega adicional, mediciones con repeticiones, casos vacíos en R5/R7/R8	Cada cambio se verificó ejecutando elixir main.exs con entradas válidas e inválidas
[COMPLETAR]	Miguel Ceballos	[COMPLETAR]	[COMPLETAR]	[COMPLETAR]
[COMPLETAR]	Juan José Ramírez	[COMPLETAR]	[COMPLETAR]	[COMPLETAR]

2. Caso de error

Caso 1: la IA afirmó algo falso sobre R8. Durante la revisión, la IA dijo que R8 salía vacío ("ningún productor entregó en los 4 tanques"). Al volver a ejecutar el programa y mirar la salida completa, R8 tenía los 10 productores: la IA había recortado la salida antes de esa sección y sacó una conclusión con información incompleta. **Cómo se detectó:** ejecutando el programa y leyendo la salida completa en vez de confiar en el resumen. **Qué se corrigió:** el problema real era otro. R8 incluía a **todos** los productores, así que no demostraba que el filtro funcionara. Se cambiaron los datos para que P09 no entregue en T2 ni T4 y P10 no entregue en T3, y ahora R8 muestra 8 de 10.

Caso 2: el documento tenía números que el programa no producía. La primera versión de este documento mostraba un mapa de R3 (% { 1 => 2100.0, 2 => 1950.0, ... }) y un tiempo de "1,2 a 2,5 ms" que no salían de nuestro programa. También describía un módulo Ranking.exs con opciones by: y order: que ya no existía: ranking/2 se había movido a Reportes con las opciones por: y orden: . **Cómo se detectó:** comparando el documento con git log , con el código y con una ejecución real. **Qué se corrigió:** todos los números de este documento salen de la ejecución que aparece en "Salida completa".

Caso 3: la medición de tiempo no servía. :timer.tc/1 sobre una sola ejecución daba 0 µs o 102 µs. Se explicó en la Parte C y se resolvió repitiendo 1000 veces.

[COMPLETAR: si la IA les propuso recursividad, structs u otra cosa fuera del alcance, ese también es un buen caso de error para agregar aquí.]

3. Reflexión

[COMPLETAR por el grupo, con sus propias palabras. Algunos puntos que salen de lo que pasó en el proyecto:]

- La IA ayudó a revisar rápido, pero se equivocó (caso 1) y dejó números sin verificar (caso 2). Todo lo que diga hay que comprobarlo ejecutando el programa.
- Los datos de prueba tienen que estar pensados para mostrar cada caso: un día que no cumple la meta, un empate (R5 día 2), un productor que no entrega en todos los tanques y un caso donde el promedio ponderado y el simple sean muy distintos.
- Separar el cálculo (puro) de la impresión (impuro) en los reportes permitió reutilizar `litros_por_dia/1` en la investigación y probar los reportes con listas vacías.